

UNIVERSITY OF CLAUDE · COURSE SYLLABUS

CSCE 647 — Image Synthesis

Light transport from first principles: the rendering equation, Monte Carlo, and a path tracer you write yourself

AT A GLANCE

Credits: 3 credit-equivalents · **Modules:** 13 · **Term:** Semester 3 (with CSCE 649 and CSCE 608)

Prerequisites: CSCE 641 Computer Graphics; CSCE 629 helpful for the acceleration-structure analysis; probability to the level of expectation and variance, developed here as needed

Effort: 12–14 hours per week · 13 modules + 2 project weeks

Deliverable: A physically based path tracer: BVH, microfacet BSDFs, multiple importance sampling, participating media, and a denoiser — written by you and validated against results whose answers are known in advance

Course description

CSCE 641 built a renderer that produces plausible images quickly. This course builds one that produces *correct* images, and defines what correct means. The difference is that every shortcut in the real-time pipeline — ambient terms, baked lighting, screen-space approximations — exists because the true answer was too expensive. Here we compute the true answer.

The subject has an unusual shape. Almost all of it follows from one equation, published by Kajiya in 1986, that fits on a single line and has no closed-form solution. The rest of the course is the consequences: the integral is infinite-dimensional, so we estimate it statistically; the estimator is noisy, so we spend most of our effort on variance reduction; and the variance is concentrated in a few hard configurations, so most published research is about those configurations specifically.

That structure makes this course unusually honest. A path tracer has a ground truth: run it long enough and it converges to the right answer, and any bug shows up as a measurable difference from a reference. Few areas of graphics let you check your work this directly, and the course leans on it constantly — **every component you write is validated against something whose answer is known analytically.**

You will write the renderer. By the end it will handle glossy surfaces, environment lighting, smoke, and skin, and you will be able to explain, for any image it produces, why each pixel has the value it does.

Learning outcomes

On finishing this course you should be able to do the following without reference material.

- 1 State the rendering equation, explain every term, and derive the path-integral form.
- 2 Implement robust ray-primitive intersection and a BVH that stands up to real scenes.
- 3 Explain Monte Carlo integration, and analyse an estimator for bias and variance.
- 4 Apply importance sampling, stratification, and low-discrepancy sequences, and measure what each one buys.
- 5 Implement microfacet BSDFs with correct sampling, and verify energy conservation.
- 6 Implement a path tracer with next-event estimation and multiple importance sampling.
- 7 Render participating media with unbiased transmittance estimation.
- 8 Compare bidirectional methods and say which light transport each one handles that path tracing does not.
- 9 Diagnose rendering artefacts from their appearance and name the mechanism.

Module schedule

One module per week for thirteen weeks, then two weeks for the final project. Each module is a slide deck, a set of lecture notes, mapped external lectures, and exercises that must run.

Wk	Module	Core question it answers
1	Radiometry: What Rendering Computes	What is the number stored in a pixel?
2	The Rendering Equation	What single equation describes all of light transport?
3	Rays and Intersection	How do you intersect a ray with geometry without it breaking?
4	Acceleration Structures	How do you avoid testing every triangle for every ray?
5	Monte Carlo Integration	How do you integrate something you cannot evaluate everywhere?
6	Sampling and Variance Reduction	How do you get the same answer with far fewer samples?
7	BSDFs and Microfacet Theory	What function describes a real surface, and how do you sample it?
8	Path Tracing and Multiple Importance Sampling	How do you actually estimate the rendering equation?
9	Lights and Sampling Them	How do you sample a light — or a million of them?
10	Participating Media	What happens when light scatters between surfaces?
11	Bidirectional Methods	What transport can a path tracer not find?
12	Denoising and Adaptive Sampling	How do you get a clean image without waiting for convergence?

Wk	Module	Core question it answers
13	Spectral Rendering, Cameras, and Getting the Image Right	How do you turn computed radiance into a correct image?

Projects

Project 1 — A correct path tracer (after Module 8)

The core renderer. Everything after Module 08 is an extension of this, so it has to be right rather than merely producing an image that looks acceptable.

Requirements

- Ray–triangle and ray–sphere intersection, watertight, with the self-intersection problem solved by offsetting along the normal rather than by a magic epsilon.
- A BVH built with the surface area heuristic, with build time and traversal statistics reported.
- Lambertian and smooth-conductor BSDFs, plus a GGX microfacet BSDF with correct importance sampling.
- A unidirectional path tracer with Russian roulette termination.
- Next-event estimation combined with BSDF sampling through multiple importance sampling, using the power heuristic.
- Stratified and low-discrepancy sampling, selectable at runtime so they can be compared.

Done means

- **The white furnace test passes:** a white Lambertian sphere in a uniform unit-radiance environment renders to exactly 1.0 everywhere, to within noise. Report the measured mean and variance.
- A Cornell box rendered to convergence and differenced against the PBRT reference, with the RMSE reported and below a stated threshold.
- A convergence plot: RMSE against sample count on log-log axes, for uniform, BSDF-sampled, NEE, and MIS integrators on the same scene. **The MIS curve must sit below the others everywhere, and you must be able to explain each gap.**
- A scene with both a small bright light and a large dim one, showing the failure of each single strategy and the success of MIS — the figure from Veach's thesis, reproduced with your own renderer.

Project 2 — Volumes, spectra, and a usable renderer (after Module 12)

Extend the core into the cases that make rendering hard, and make the result fast enough to iterate with.

Requirements

- Homogeneous and heterogeneous participating media with unbiased transmittance by delta tracking.
- The Henyey–Greenstein phase function, importance sampled.
- An environment light importance sampled by luminance, with the 2D CDF built at load time.
- Spectral rendering with hero wavelength sampling, demonstrated on a dispersive material.
- Adaptive sampling driven by a per-pixel variance estimate.
- A denoiser — at minimum an edge-aware à-trous filter guided by albedo and normal buffers.

Done means

- A heterogeneous medium rendered with delta tracking, shown to match a brute-force ratio-tracking reference to within noise.
- A dispersion render that produces the correct spectral ordering, beside the same scene in RGB showing the artefact RGB produces.
- A denoised image beside its noisy input and a converged reference, with RMSE for all three and an honest note on what the denoiser destroyed.
- An equal-time comparison: your renderer against PBRT on the same scene and hardware. **You will lose. Report by how much and say where the time goes.**

Assessment

Nobody grades this. The standard is therefore mechanical, and you apply it to yourself honestly or the credential is worth nothing.

Component	Weight	Standard
Module exercises	40%	Code runs and produces the specified output. Not 'I understand how I would do it.'
Projects	45%	In the portfolio repository, with a README showing results and a written note on what was hard.
Self-checks	15%	Answered in writing from memory. Recognition is not knowledge; writing exposes the difference.

Required materials

All free. Nothing in this course requires a purchase.

Resource	Role
Physically Based Rendering, 4th ed. — free online	The primary text, and the most complete free resource in graphics. It is a literate program: the book is the renderer. Read the chapter before each module and read the code with it.
Ray Tracing in One Weekend series (free)	Three short books that get a working path tracer on screen fast. Do the first one before Module 03 — getting something rendering early makes the theory concrete.
TU Wien — Rendering / Light Transport (free video)	Károly Zsolnai-Fehér's full graduate course. Excellent on the intuition behind Monte Carlo and on the comparative strengths of bidirectional methods.

Resource	Role
Eric Veach — Robust Monte Carlo Methods for Light Transport (thesis, free)	Where multiple importance sampling, BDPT, and Metropolis light transport come from. Chapter 9 on MIS is the clearest exposition that exists and is worth reading in the original.
Scratchapixel — rendering lessons (free)	Slower and more elementary than PBRT, with the algebra written out. Useful when a PBRT derivation moves too fast.
Benedikt Bitterli — rendering resources and test scenes (free)	Scenes in PBRT format with reference renders. Your validation set from Module 08 onward.

Tooling

- **C++17** and nothing else for the core. You need the performance, and this is a course where a 20× speed difference changes which experiments you can run.
- **No rendering library.** Intersection, sampling, and integration are the course. You may use a linear-algebra header and an image writer.
- **OpenEXR** for output. Rendering is high-dynamic-range work and writing 8-bit PNG from the start will hide errors that matter.
- **PBRT v4** installed as a *reference*, not a dependency. Render the same scene in both and difference the images — this is the single most effective debugging tool in the course.
- **A validation suite:** the white furnace test, a Cornell box with a known analytic solution for direct lighting, and a convergence plot script. Build it in Module 05 and never delete it.
- **A portfolio repository** with every image the course produces, its sample count, and its render time.

Academic honesty, restated for a solo student

There is no one to cheat but yourself, which makes it easier, not harder. Two rules: copy code only after you have written a worse version yourself, and never let a working result stand in for an understood one. When you paste something you do not understand, write a TODO naming what you do not understand, and go back to it.